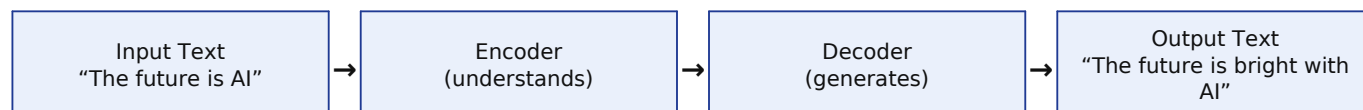


TRANSFORMERS

Detailed Notes & Revision Guide

From Attention to Intelligence — The Core Architecture Behind Modern AI



Attention + Context + Better Representations = Smarter AI

Subtopics Covered

#	Topic	Covers
1	Transformer Architecture & Components	Encoder, Decoder, Transformer block
2	Multi-Head Attention	Why multiple heads, parallel attention, different relationships, combining heads
3	Transformer Encoder	Encoder block, self-attention, FFN, residual connections, layer norm
4	Transformer Decoder	Masked self-attention, cross-attention, FFN, residual connections, layer norm
5	Feed-Forward Network & Normalization	Position-wise FFN, activation functions, layer normalization, residuals
6	Training Transformers	Forward propagation, loss function, backpropagation, optimizers, LR scheduling, teacher forcing
7	Advanced Transformers & Interview Concepts	KV cache, attention complexity, long-context, Flash Attention, MoE, quantization, optimization, interview Q&A
8	BERT, GPT & Transformer Variants	BERT (encoder), GPT (decoder), T5 (encoder-decoder), RoBERTa & others, when to use each
9	Transformers in LLMs & Generative AI	LLMs, text generation, context window, prompting, autoregressive generation, applications
10	Practical Insights	Key differences, advantages, limitations, real-world use cases
11	Important Formulas & Concepts	Attention formula, softmax, positional encoding, complexity
12	Diagrams & Visuals	Architecture diagrams, flow charts, attention mechanism, model comparisons
13	Quick Revision	Key takeaways, do's & don'ts, exam/interview points

Notes source: @techyy_bandaa (Instagram). Retyped for clarity; parts of the original pages that were hidden behind on-screen overlays have been reconstructed from context.

Transformers — Page 1

SUBTOPIC: Introduction to Transformers

1. What are Transformers?

Definition

- Transformers are a type of **deep learning model** designed to work with **sequential** data (like text, code, etc.).
- They are based on the **self-attention mechanism**, which helps the model understand the context of words in a sequence.
- Unlike RNNs, transformers do not process tokens one by one. They can look at all tokens in **parallel**.

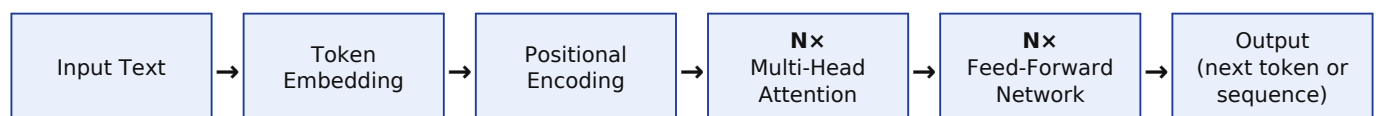
Key idea: Self-Attention

- Allows the model to focus on relevant words in the input sequence.
- Captures long-range dependencies effectively.

Example

- In the sentence “The cat sat on the mat”, when predicting “mat”, the model can directly look at “cat”, “sat”, “on”, “the” (all tokens) and decide which ones are important.

Transformer Architecture (High Level)



Main Components

- Token Embedding** — converts tokens to vectors.
- Positional Encoding** — adds position information.
- Multi-Head Attention** — focuses on different parts of the input simultaneously.
- Feed Forward Network** — applies non-linearity and learns complex patterns.
- Layer Normalization & Residual Connection** — helps with stable training.

★ Remember

- Transformers are more parallelizable and scalable than RNNs, making them ideal for large-scale training.

2. Why Transformers were introduced?

Motivation

- RNNs and LSTMs struggled with long-range dependencies.
- Training was slow due to sequential processing.
- Limited ability to capture global context.
- Transformers solve these issues using self-attention and parallel computation.

Key reasons

- Better handling of long sequences.
- Faster training (parallelizable).
- Improved performance in NLP tasks.
- Scales well with large data and model sizes.

3. RNN / LSTM Limitations

RNN Limitations

- Vanishing / exploding gradient problem.
- Struggles with long-term dependencies.
- Sequential processing → slow training.
- Limited memory of past information.

LSTM Limitations

- Solves vanishing gradient to some extent, but still not perfect for very long sequences.
- More complex architecture (more parameters).
- Slower than transformer due to sequential nature.

RNN vs LSTM vs Transformer

Feature	RNN	LSTM	Transformer
Long-term dependencies	Poor	Better	Excellent
Training speed	Slow	Slow	Fast (parallel)
Memory	Limited	Better	Global context
Architecture	Simple	More complex	Attention-based
Scalability	Low	Medium	High

4. Transformer Applications

- **Natural Language Processing (NLP)** — BERT, GPT, T5, etc.
- **Machine Translation** — Google Translate, etc.
- **Text Summarization** — news, articles, documents.
- **Question Answering** — chatbots, QA systems.
- **Text Generation** — content creation, code generation.
- **Computer Vision** — Vision Transformers (ViT).
- **Speech Processing** — speech-to-text, text-to-speech.
- **Multimodal AI** — image + text, audio + text, etc.

Real-world examples

- ChatGPT (GPT series)
- Google BERT (search, NLP)
- DALL·E (image generation)
- → Used in almost every modern AI application today!

Transformers — Page 2

SUBTOPIC: Transformer Architecture

1. What is Transformer Architecture?

- The overall structure of the model, showing how different components (encoder, decoder, attention, feed-forward layers, etc.) work together.
- It defines the flow of information from input to output.
- It is based on the **self-attention** mechanism instead of recurrent or convolutional layers.

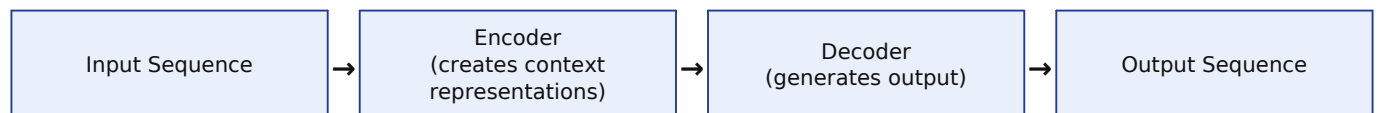
Key idea

- Use self-attention to capture global context and process the entire sequence in parallel.

2. Encoder-Decoder Architecture

- Used in sequence-to-sequence tasks (e.g., translation, summarization, question answering).
- **Encoder** reads the input sequence and learns a contextual representation.
- **Decoder** generates the output sequence, using the context from the encoder.

Flow



Example

- Input: "The cat sits"
- Output: "Le chat est assis" (French translation)

3. Input → Encoder → Decoder → Output

Step-by-step flow

1. Input tokens are converted to embeddings and positional encodings are added.
2. The **Encoder** processes the input using self-attention and feed-forward layers.
3. The **Decoder** attends to the encoder output (cross-attention) and generates the output tokens.
4. Output is converted to vocabulary probabilities using a linear layer + softmax.

★ Important

- Encoder and decoder have similar structure (stacked layers).
- Decoder has an extra masked self-attention layer.
- Positional encoding is needed to retain word order.

4. Main Components

① Multi-Head Attention

- Captures different types of relationships.
- Uses multiple attention heads in parallel.

② Feed Forward Network (FFN)

- Two linear layers with ReLU.
- Applied to each position separately.

③ Add & Norm

- Residual connection + Layer Normalization.
- Helps in stable and faster training.

④ Positional Encoding

- Adds information about token position.
- Uses sin/cos functions (or learned embeddings).

5. Original Transformer Architecture

Encoder (N×), bottom → top	Decoder (N×), bottom → top
Input → Input Embedding + Positional Encoding → Multi-Head Self-Attention → Add & Norm → Feed Forward Network → Add & Norm → (encoder output)	Output (shifted right) → Output Embedding + Positional Encoding → Masked Multi-Head Self-Attention → Add & Norm → Multi-Head Cross-Attention (uses encoder output) → Add & Norm → Feed Forward Network → Add & Norm

Where

- N = number of layers (e.g., 6 in base model).
- Each layer has 2 sub-layers (attention + FFN); the decoder has 3 (masked self-attention, cross-attention, FFN).
- Residual connections + LayerNorm after each sub-layer.
- Positional encoding is added to both encoder and decoder inputs.

★ Key Points

- Encoder is used to understand the input.
- Decoder generates the output step by step.
- Masked self-attention prevents looking at future tokens.
- Cross-attention helps decoder focus on relevant parts of encoder output.
- Works well for parallelization and long-range dependencies.

Transformers — Page 3

SUBTOPIC: Tokenization & Input Representation

1. Tokens and Vocabulary

- Text is broken into smaller units called **tokens**.
- Each token is a word, subword or character.
- The complete set of unique tokens is called the **vocabulary**.
- Vocabulary is fixed before training and used for converting text to token IDs.

Key Points

- Tokens can be words, subwords or characters.
- Vocabulary size is usually 10K – 100K+.
- Special tokens like [PAD], [UNK], [CLS], [SEP] are also part of the vocabulary.

Example (word level)

- “The cat is on the mat” → [The] [cat] [is] [on] [the] [mat]
- Vocabulary = {The, cat, is, on, the, mat, ...}

★ Remember

- Larger vocabulary gives more flexibility but requires more memory and computation.

2. Word / Subword Tokenization

- Word tokenization** splits text into words.
- Subword tokenization** (e.g. BPE, WordPiece) splits rare words into smaller pieces.
- Helps handle unknown words and reduces vocabulary size.

Common methods

- Word-level (simple, but large vocab).
- Subword-level (BPE, WordPiece, SentencePiece).
- Character-level (rarely used in modern LLMs).

Method	Out of vocab?	Vocab size	Example
Word-level	High	Large	“unbelievable”
BPE	Low	Medium	“un ##believ ##able”
WordPiece	Low	Medium	“un ##believ ##able”
Character-level	Very low	Very large	“u n b e l i e v a b l e”

Example (BPE / WordPiece)

- “unbelievable” → [un] [##believ] [##able]
- Subword tokenization is most commonly used in most modern LLMs.

3. Token IDs

- Each token from the vocabulary is mapped to a unique integer ID.
- These IDs are used as input to the model.
- Special tokens have fixed IDs (e.g. [PAD] = 0, [UNK] = 1).

Token	ID
[PAD]	0
[UNK]	1
The	2

Token	ID
cat	3
is	4
...	...

Key Points

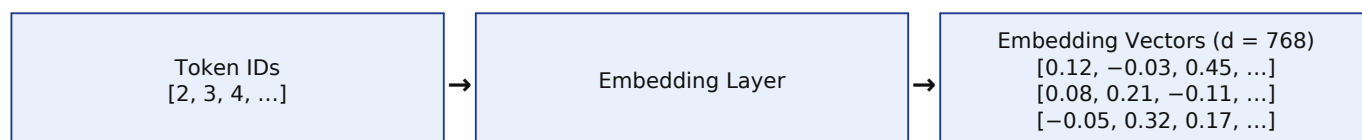
- Token IDs are integers.
- Range: 0 to vocab_size – 1.
- Used for embedding lookup.
- Padding token helps in batching sequences.

4. Embeddings

- Embeddings convert token IDs into **dense vectors**.
- Capture semantic meaning and context.
- Learned during training.

Key Points

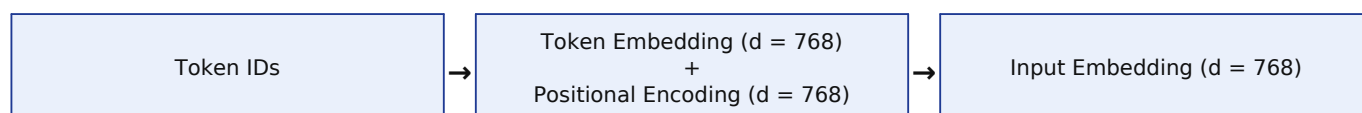
- Embedding dimension = 768 (BERT base), 1024 (BERT large).
- Tokens with similar meaning have similar embeddings.
- Can be initialized randomly or with pre-trained vectors.



5. Input Embeddings

- Combines token embeddings with positional information.
- Helps the model understand the order of tokens.
- Input to the first encoder layer.

Input Embedding = Token Embedding + Positional Encoding



★ Remember

- Positional encoding can be sinusoidal or learned.
- Final input shape: (batch_size, seq_len, d_model).

Transformers — Page 4

SUBTOPIC: Positional Encoding

1. Why position information is needed

What is the problem?

- Transformer uses self-attention, which is **permutation invariant** (doesn't consider the order of tokens).
- Without position information, the model cannot distinguish between “dog bites man” and “man bites dog”.
- Hence, we need to inject the position of each token into the input representation.

Key idea

- Add positional information to the input embeddings so the model knows the relative / absolute position of each token in the sequence.

2. Sinusoidal positional encoding

- Uses sine and cosine functions of different frequencies.
- Allows the model to learn relative positions easily.

$$\begin{aligned} \text{PE}(\text{pos}, 2i) &= \sin(\text{pos} / 10000^{2i/d_{\text{model}}}) \\ \text{PE}(\text{pos}, 2i+1) &= \cos(\text{pos} / 10000^{2i/d_{\text{model}}}) \end{aligned}$$

- pos** = position of the token; **i** = dimension; **d_model** = model dimension

Properties

- Produces unique encoding for each position.
- Allows the model to easily learn relative positions (using linear operations).
- Works well for longer sequences (extrapolates).

Example (d_model = 6)

Position	Dim 0 (sin)	Dim 1 (cos)	Dim 2 (sin)	Dim 3 (cos)	Dim 4 (sin)	Dim 5 (cos)
0	0.000	1.000	0.000	1.000	0.000	1.000
1	0.841	0.540	0.046	0.999	0.002	1.000
2	0.909	-0.416	0.093	0.996	0.004	1.000
3	0.141	-0.990	0.139	0.990	0.006	1.000
...	...	...	...	...	...	...

(Values are rounded for simplicity; the last two columns above are recomputed from the formula because the printed values were partly unclear.)

3. Learned positional embeddings

- Position is represented using a **learnable embedding vector** (like word embeddings).
- A separate embedding is maintained for each position (up to a maximum length).

$$\mathbf{X} = \text{Token Embeddings} + \text{Position Embeddings}$$

(Position embeddings are learned parameters)

Advantages

- Can learn task-specific position representations.
- Often gives better performance in practice.

Disadvantages

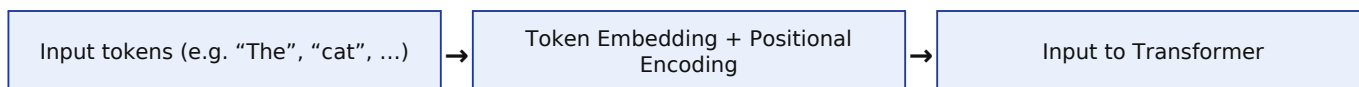
- Doesn't generalize well to longer sequences (limited to max length seen during training).
- Uses more parameters.

Example: learnable position embeddings (lookup table)

Position 0	Position 1	Position 2	...	Position n
P0	P1	P2	...	Pn

4. Position information in Transformers

- Positional encodings (sinusoidal or learned) are added to the input embeddings at the start.
- The combined representation is passed to the encoder and decoder.
- This allows the model to use position information while computing attention.



Key points

- Both encoder and decoder use positional encoding.
- In some models, relative positional encoding is used (e.g., Transformer-XL, T5).
- Position information helps with understanding order, structure and long-range dependencies.

Transformers — Page 5

SUBTOPIC: Self-Attention Mechanism

1. What is attention?

- Attention is a mechanism that allows the model to focus on relevant parts of the input sequence while processing a token.
- It helps in capturing relationships between different tokens, no matter how far apart they are.
- In Transformers, **self-attention** is used, where the attention is computed within the same sequence (not from an external source).

★ Key idea

- Instead of processing tokens one by one (like RNNs), attention lets the model look at all tokens in parallel and decide which ones are important.

2. Query (Q)

- Query represents the current token (or position) for which we want to find relevant information.
- It is compared with all keys to calculate attention scores.
- Shape: (seq_len, d_model)

Example

- For the word “cat”, its query vector is a learned representation that is used to find relevant keys.

3. Key (K)

- Key represents the other tokens in the sequence.
- It is used to compare with the query.
- Each token has its own key vector.
- Shape: (seq_len, d_model)

Example

- All tokens (e.g. “The”, “cat”, “is”, ...) have key vectors. The query is matched with these keys to get attention scores.

4. Value (V)

- Value contains the actual information of each token.
- After computing attention scores, the values are combined to get the final representation.
- Shape: (seq_len, d_model)

Example

- For each token, the value vector holds the information to be passed to the next layer.

5. Attention calculation

Step 1: Compute Q, K, V

- Apply 3 separate linear transformations to the input embeddings (X).

$$Q = X \cdot W_Q \quad K = X \cdot W_K \quad V = X \cdot W_V$$

- W_Q , W_K , W_V are learnable weight matrices.
- Input: X (seq_len $\times$ d_model). Output: Q , K , V (seq_len $\times$ d_model).

Step 2: Compute attention scores

- Use scaled dot-product to get similarity between query and key (d_k = dimension of keys/queries).

$$\text{Attention scores} = Q \cdot K^T / \sqrt{d_k}$$

Step 3: Apply softmax

- Convert scores into probabilities (so they sum to 1).

$$\text{Attention weights} = \text{softmax}(Q \cdot K^T / \sqrt{d_k})$$

Step 4: Get weighted sum of values

- Multiply attention weights with the value vectors.

$$\text{Output} = \text{Attention weights} \times V$$

Simple example (1 token attending to 4 tokens: The, cat, is, cute)

- Input tokens → Q, K, V → Attention Scores → Softmax → Weights → Weighted sum of V → Output representation.
- Each token can attend to all tokens in the sequence and get a context-aware representation.

★ Key points

- Q finds relevant tokens.
- K is used for comparison.
- V contains actual information.
- Scaling ($1/\sqrt{d_k}$) prevents large dot products.
- Softmax converts scores to probabilities.
- Self-attention is computed in parallel for all tokens.

Transformers — Page 6

SUBTOPIC: Scaled Dot-Product Attention

1. What is Scaled Dot-Product Attention?

Definition

- A mechanism used in Transformers to compute attention scores between tokens in a sequence.
- It helps the model decide which tokens are more relevant while processing a token.
- Uses dot product of query and key, with scaling and softmax to get attention weights.

Key idea

- Compute similarity between query and keys, scale it, apply softmax and use the weights to get a weighted sum of values.

2. Q, K, V Matrices

- **Q (Query):** represents what the current token is looking for.
- **K (Key):** represents what each token contains.
- **V (Value):** represents the actual information to be aggregated.

Shapes

- If input sequence length = n and model dimension = d_{model} , then $Q, K, V \in \mathbb{R}^{n \times d_{\text{model}}}$.

Example

- For a sentence with 4 tokens and $d_{\text{model}} = 64$, Q, K, V each have shape 4×64 .

3. Dot Product

- Measures similarity between two vectors. Higher value $\rightarrow$ more similar.

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1..d} \mathbf{a}_i \cdot \mathbf{b}_i$$

In attention

- We compute $Q \cdot K^T$ (query with all keys).
- Result is a similarity matrix of shape $n \times n$.

Key points

- The score shows how much each token attends to other tokens.
- Higher dot product $\rightarrow$ higher attention score (before scaling).

4. Scaling

Why scaling?

- Dot products can get very large as d_{model} increases.
- This pushes softmax into regions with very small gradients.
- To avoid this, we scale the dot product by $\sqrt{d_k}$.

$$\text{Scaled Dot Product} = Q \cdot K^T / \sqrt{d_k}$$

- d_k is the dimension of key vectors (usually d_{model} / h).

Key point

- Scaling keeps the values in a range where softmax works properly and gradients are stable.

5. Softmax

- Converts the scaled scores into probabilities.
- Ensures the weights sum to 1 and are between 0 and 1.

$$\text{Softmax}(x_i) = e^{x_i} / \sum_{j=1..n} e^{x_j}$$

Properties

- Output values are in (0, 1).
- Sum of all values = 1.
- Differentiable (important for training).

Why softmax?

- Normalizes the scores.
- Gives more focus to relevant tokens.

6. Attention Score Formula

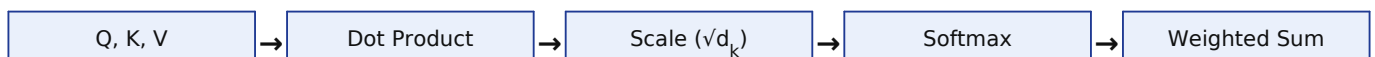
$$\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$$

Steps

1. Compute dot product: $Q \cdot K^T$
2. Scale by $\sqrt{d_k}$
3. Apply softmax to get attention weights
4. Multiply weights with V
5. Get final output (weighted sum)

Example (simplified)

- $Q = [1, 0], K = [1, 0], V = [2, 3], d_k = 2$
- 1. $Q \cdot K^T = 1$
- 2. Scaled = $1/\sqrt{2} \approx 0.707$
- 3. $\text{Softmax}(0.707) = 1$ (single token example)
- 4. Output = $1 \times V = [2, 3]$



Transformers — Page 7

SUBTOPIC: Multi-Head Attention

1. Why multiple attention heads?

What is the idea?

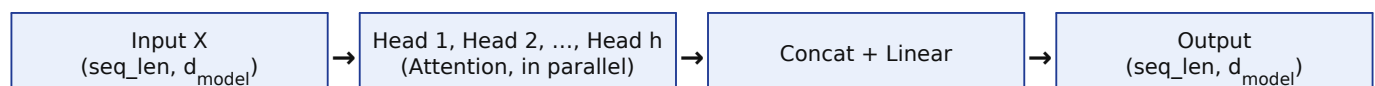
- A single attention head has limited capacity to focus on different types of relationships in the input.
- Using multiple heads allows the model to attend to information from different representation subspaces at the same time.
- Each head can learn different patterns (e.g. syntactic, semantic, positional etc.).

Key points

- Improves model's ability to capture diverse features.
- Helps in learning richer representations.
- Leads to better performance in many NLP tasks.

2. Parallel attention

- All attention heads are computed in parallel (not one after another).
- The input is projected into multiple heads simultaneously.
- This makes the computation efficient and faster.



Key points

- Heads are computed in parallel using matrix operations.
- Uses parallelizable GPU/TPU computation.
- Reduces training time compared to sequential attention.

3. Different relationships captured by each head

- Each head learns to focus on different types of relationships between tokens.
- Some heads may capture syntactic relations (e.g. subject-verb).
- Some heads may capture semantic relations (e.g. coreference).
- Some heads may focus on positional or local patterns.

Examples

- Head 1: captures subject-verb agreement.
- Head 2: captures coreference (e.g. "it" → "dog").
- Head 3: focuses on local context (nearby words).
- Head 4: captures long-range dependencies.

Illustration — "The cat that chased the mouse was hungry."

Head 1 (syntactic)	Head 2 (coreference)	Head 3 (local)	Head 4 (long-range)
The cat that chased ...	cat ↔ that ↔ chased ... mouse	chased the mouse	The ... hungry

Key points

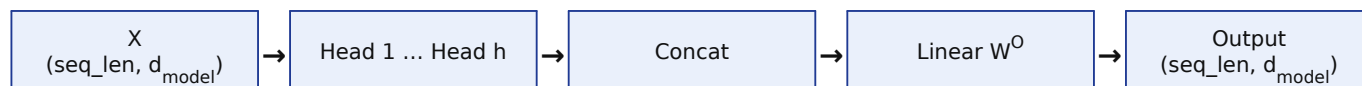
- Each head has its own set of learned parameters.
- Heads work on different subspaces of the input.
- Combined, they give a richer understanding of the context.

4. Combining attention heads

- 1. Compute attention for each head in parallel.
- 2. Concatenate the outputs from all heads.
- 3. Apply a linear transformation to get the final output.

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h) \mathbf{W}^O$$
$$\text{head}_i = \text{Attention}(\mathbf{Q} \mathbf{W}_i^Q, \mathbf{K} \mathbf{W}_i^K, \mathbf{V} \mathbf{W}_i^V)$$

- h = number of heads; $\mathbf{W}_i^Q, \mathbf{W}_i^K, \mathbf{W}_i^V$ = separate projection matrices for each head; $\mathbf{W}^O$ = output projection matrix.



Key points

- Concatenation combines all head outputs.
- Linear layer (W^O) projects back to model dimension.
- Helps the model integrate information from all heads.

★ Quick takeaway

- Multi-head attention = multiple parallel attention heads + concatenation + linear projection.
- → Captures diverse relationships and improves performance.

Transformers — Page 8

SUBTOPIC: Transformer Encoder

1. What is Transformer Encoder?

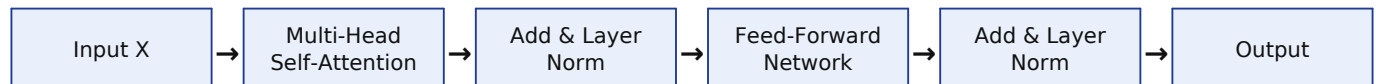
- The part of the Transformer model that reads the input sequence and converts it into a sequence of contextualized representations.
- It uses self-attention and feed-forward layers to understand the meaning of each token in the context of the entire sequence.
- The encoder is composed of multiple identical layers (stacked one after another).

Key idea

- The encoder captures the context from the input sequence and produces rich representations (embeddings) for each token.

2. Encoder Block

- The encoder is built using a repeating block called the encoder layer.
- Each layer has 2 main sub-layers: (1) Multi-Head Self-Attention, (2) Feed-Forward Network (MLP).
- Both sub-layers are wrapped with residual connections and layer normalization.



3. Self-Attention

- Allows each token to attend to all tokens in the input sequence.
- Computes attention scores to find how much focus each token should give to other tokens.
- Helps in capturing long-range dependencies and contextual meaning.

Key points

- Each token attends to all tokens (including itself).
- Attention scores are computed using Q, K, V.
- Multi-head attention uses multiple attention heads to capture richer representation.

4. Feed-Forward Network

- After self-attention, each token is passed through a feed-forward network (MLP) independently.
- It consists of two linear transformations with a ReLU activation in between.

$$\text{FFN}(\mathbf{x}) = \max(0, \mathbf{x}\mathbf{W}_1 + \mathbf{b}_1) \mathbf{W}_2 + \mathbf{b}_2$$

- $\mathbf{W}_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$, $\mathbf{W}_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$
- d_{ff} is usually $4 \times d_{\text{model}}$ (e.g. 2048 if $d_{\text{model}} = 512$).

5. Residual Connections

- Residual connections add the input of each sub-layer to its output.
- Helps in better gradient flow and avoids the vanishing gradient problem.

$$\text{Output} = \text{Layer}(\mathbf{x}) + \mathbf{x}$$

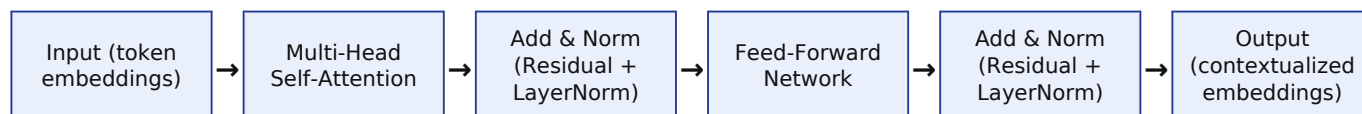
6. Layer Normalization

- Normalizes the activations of each layer across the features.
- Stabilizes training and improves convergence.

$$\text{LN}(\mathbf{x}) = (\mathbf{x} - \boldsymbol{\mu}) / \sqrt{(\boldsymbol{\sigma}^2 + \epsilon)} \cdot \boldsymbol{\gamma} + \boldsymbol{\beta} \text{ (for a single token)}$$

- μ = mean; σ^2 = variance; γ, β = learnable parameters; ϵ = small constant.

7. Encoder Layer Flow



Key points

- The encoder has N identical layers (e.g. 6 layers).
- Each layer maintains the same dimension (d_{model}).
- Position encoding is added to input embeddings (before first layer).
- The final output is a sequence of contextual representations.

Transformers — Page 9

SUBTOPIC: Transformer Decoder

1. Transformer Decoder

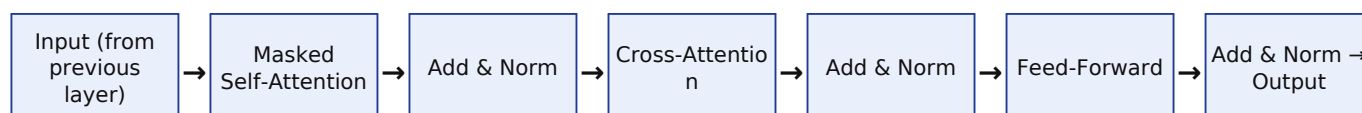
- The part of the Transformer model that generates the output sequence (e.g. translation, text generation) **one token at a time**.
- It uses masked self-attention, cross-attention and feed-forward layers.
- Each decoder block is stacked multiple times (N times) to form the full decoder.

Key idea

- The decoder attends to both the already generated tokens and the encoder's output to produce the next token.

2. Decoder Block

- The decoder block is the building block of the decoder.
- It has 3 main sub-layers: (1) Masked Self-Attention, (2) Cross-Attention, (3) Feed-Forward Network.
- Each sub-layer is followed by a residual connection and layer normalization.



- The output of one block is passed as input to the next decoder block; stacked N times.

Key points

- Uses masked self-attention (to prevent looking ahead).
- Uses cross-attention (to attend to encoder output).
- Helps generate tokens in a left-to-right manner.

3. Masked Self-Attention

- Similar to regular self-attention, but with a mask.
- The mask prevents the model from attending to future tokens.
- Ensures auto-regressive (left-to-right) generation.

Masking

- Uses a triangular matrix where future positions are set to $-\infty$ (or a very large negative value).

Example (mask matrix for seq len = 4)

	1	2	3	4
1	0	$-\infty$	$-\infty$	$-\infty$
2	0	0	$-\infty$	$-\infty$
3	0	0	0	$-\infty$
4	0	0	0	0

- $-\infty$ = very large negative value (ignored in softmax).

Key points

- Allows each position to attend only to previous positions (including itself).
- Maintains the order of the sequence.

4. Cross-Attention

- Used to attend to the encoder's output (source sequence).
- Helps the decoder focus on relevant parts of the input.
- The **queries** come from the decoder, while **keys and values** come from the encoder output.

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}(\mathbf{Q} \cdot \mathbf{K}^T / \sqrt{d_k}) \cdot \mathbf{V}$$

- $\mathbf{Q}$ = decoder's queries; $\mathbf{K}, \mathbf{V}$ = encoder's keys and values; d_k = dimension of keys/queries.

Key points

- Connects decoder with encoder output.
- Helps in understanding the input context.

5. Feed-Forward Network

- Same as in the encoder.
- Applies two linear transformations with a ReLU in between.
- Works independently on each position.

$$\text{FFN}(\mathbf{x}) = \max(0, \mathbf{x}\mathbf{W}_1 + \mathbf{b}_1) \mathbf{W}_2 + \mathbf{b}_2$$

- $\mathbf{W}_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$; $\mathbf{W}_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$; d_{ff} is usually $4 \times d_{\text{model}}$ (e.g. 2048 if $d_{\text{model}} = 512$).

6. Autoregressive Generation

- The decoder generates tokens one by one, in a sequential way.
- Each new token is conditioned on the previously generated tokens.
- Uses masked self-attention to maintain causality.
- Steps: start with a <BOS> token → predict the next token using the decoder → add it to the sequence and repeat.

Example

- Input: <BOS> I love NLP
- Output (step by step): <BOS> → I → love → NLP → <EOS>

Key points

- Ensures proper sequence generation.
- Used in tasks like translation, summarization, text generation.
- Often combined with beam search or greedy decoding.

Transformers — Page 10

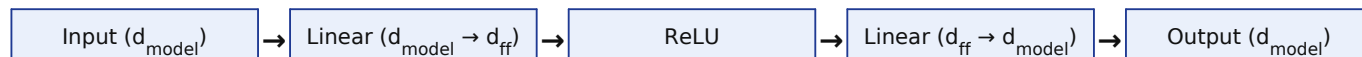
SUBTOPIC: Feed-Forward Neural Network & Normalization

1. Position-wise Feed-Forward Network (FFN)

- A simple 2-layer fully connected network applied independently to each position (token).
- It increases the model's capacity and introduces non-linearity.
- Same weights are used across all positions.

$$\text{FFN}(\mathbf{x}) = \max(0, \mathbf{x}\mathbf{W}_1 + \mathbf{b}_1) \mathbf{W}_2 + \mathbf{b}_2$$

- $\mathbf{W}_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$, $\mathbf{W}_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$; d_{ff} is usually $4 \times d_{\text{model}}$ (e.g. 2048 if $d_{\text{model}} = 512$).



2. Activation Functions

Function	Formula	Properties
ReLU	$f(x) = \max(0, x)$	Simple; fast; helps with vanishing gradient
GELU	$f(x) = x \cdot \Phi(x)$ (Φ = Gaussian CDF)	Smoother than ReLU; better performance in many NLP tasks
Tanh	$f(x) = (e^x - e^{-x}) / (e^x + e^{-x})$	Output in $(-1, 1)$; zero centered

★ Note

- GELU is commonly used in modern Transformer models (e.g. BERT, GPT).

3. Layer Normalization

- Normalizes the activations across the feature dimension for each token (instead of across the batch).
- Helps stabilize training and makes the model less sensitive to learning rate.

$$\text{LN}(\mathbf{x}) = (\mathbf{x} - \boldsymbol{\mu}) / \sqrt{(\boldsymbol{\sigma}^2 + \epsilon)} \cdot \boldsymbol{\gamma} + \boldsymbol{\beta}$$

- $\boldsymbol{\mu}$ = mean (across features); $\boldsymbol{\sigma}^2$ = variance (across features); $\boldsymbol{\gamma}$, $\boldsymbol{\beta}$ = learnable parameters; ϵ = small constant.

Key points

- Applied after attention and FFN (with residual connection).
- Helps with stable and faster training.
- Unlike Batch Norm, it works well with small batch sizes.

4. Residual Connections

- Adds the input of a sub-layer to its output.
- Helps in better gradient flow and prevents vanishing gradient problem.
- Allows training of deeper models.

$$\text{Output} = \text{Layer}(\mathbf{x}) + \mathbf{x}$$

- $\text{Layer}(\mathbf{x})$ = output of sub-layer (e.g. attention or FFN); $\mathbf{x}$ = input to the sub-layer.

★ Note

- Residual + LayerNorm (in practice) is used after each sub-layer.

5. Transformer Block Flow

How it all fits together



Key takeaways

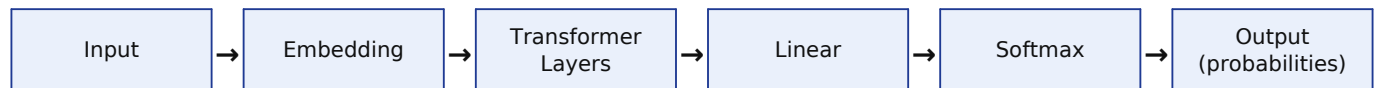
- FFN adds non-linearity and capacity.
- Activation functions control the learning behavior.
- Layer Norm stabilizes training (feature-wise).
- Residual connections help gradient flow.
- Together, they make the Transformer deep and trainable.

Transformers — Page 11

SUBTOPIC: Training Transformers

1. Forward Propagation

- The process of passing the input through the model to get the output.
- In a Transformer, input tokens are converted to embeddings, then passed through multiple layers (encoder/decoder).
- The final layer produces logits, which are converted to probabilities using softmax.



Key points

- Each layer consists of self-attention, add & norm, and feed-forward network.
- During training, we compute the loss using the output probabilities and target tokens.

2. Loss Function

- Measures how well the model's predictions match the actual target values.
- For language modeling, we use **Cross-Entropy Loss**.

$$L = - (1/N) \sum_{i=1..N} \sum_{v=1..V} y_{i,v} \cdot \log(p_{i,v})$$

- N = number of tokens in the batch; V = vocabulary size; $y_{i,v} = 1$ if token v is the target, else 0; $p_{i,v}$ = predicted probability for token v.

Key points

- Lower loss means better predictions.
- In practice, we use label smoothing to prevent overfitting.

3. Backpropagation

- Calculates gradients of the loss w.r.t. model parameters using the chain rule.
- It helps update the weights to reduce the loss.
- In Transformers, gradients flow through attention, FFN, and embedding layers.

Key steps

1. Compute loss and output gradients.
2. Backpropagate through linear layers, attention and FFN.
3. Update parameters using optimizer.

Key points

- Uses chain rule of derivatives.
- Efficiently computes gradients for large models.

4. Optimizers

- Optimizers update model parameters based on gradients.
- Common optimizers: Adam, AdamW, SGD.

Adam update rule

$$\begin{aligned} \mathbf{m}_t &= \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t \\ \mathbf{v}_t &= \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2 \\ \boldsymbol{\theta}_t &= \boldsymbol{\theta}_{t-1} - \alpha \cdot \hat{\mathbf{m}}_t / (\sqrt{\hat{\mathbf{v}}_t} + \epsilon) \end{aligned}$$

- $\mathbf{g}_t$ = gradient; $\mathbf{m}_t, \mathbf{v}_t$ = moment estimates; β_1, β_2 = decay rates; α = learning rate; ϵ = small constant.

Key points

- Adam is widely used for training Transformers.
- AdamW decouples weight decay for better generalization.

5. Learning-Rate Scheduling

- Adjusts the learning rate during training.
- Helps the model converge faster and more stably.
- Common strategies: warmup + decay, linear decay, cosine decay.

Example: Warmup + Cosine Decay

- The learning rate rises linearly during warmup, then falls smoothly along a cosine curve.

$$\text{lr} = \text{lr}_{\min} + \frac{1}{2} (\text{lr}_{\max} - \text{lr}_{\min}) \times (1 + \cos(\pi t / T))$$

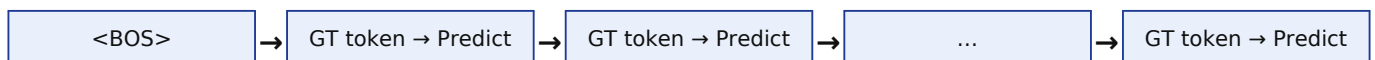
- t = current step; T = total steps; $\text{lr}_{\max}, \text{lr}_{\min}$ = max/min learning rate.

Key points

- Warmup prevents instability at the start.
- Cosine decay gives better final performance.

6. Teacher Forcing

- During training, we feed the ground-truth token as input to the decoder (instead of the model's prediction).
- Helps the model learn faster and reduces error accumulation.



Key points

- Used only during training.
- During inference, we use the model's own predictions.
- Important for stable and faster training.

★ Remember

- Teacher forcing is a training technique, not used during inference.

Transformers — Page 12

SUBTOPIC: Pre-training & Fine-Tuning

1. Pre-training

- The process of training a large model on a massive amount of unlabeled data.
- The model learns general language patterns, world knowledge and useful representations.
- It is done once and provides a strong starting point for many downstream tasks.

Key points

- Uses large-scale, diverse data (e.g. web text).
- Trains with self-supervised objectives (e.g. masked LM).
- Results in a general-purpose model (e.g. BERT, GPT).

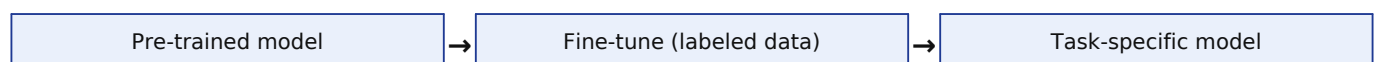


2. Fine-tuning

- Uses the pre-trained model to a specific downstream task using labeled data.
- The model's weights are updated (usually with a smaller learning rate).
- It helps the model perform well on the target task without training from scratch.

Key points

- Uses task-specific labeled data.
- Only a few layers or all layers are updated.
- Faster and better performance than training from scratch.



3. Transfer Learning

- Uses knowledge gained from one task (source) to improve performance on another task (target).
- In transformers, this is often done using pre-trained models and fine-tuning on a new task.

Key points

- Works well when target data is limited.
- Saves time and computational cost.
- Improves generalization.

4. Self-Supervised Learning

- Learns from the data itself without explicit labels.
- The model creates its own training signals (pseudo-labels).
- Common methods: masked language modeling, next sentence prediction, contrastive learning.

Key points

- Uses unlabeled data.
- Learns useful representations.
- Forms the basis for most modern LLMs.

Example: Masked Language Modeling (MLM)

- Input: "The [MASK] is great" → Predict masked token → Output: "cat"

5. Task-Specific Models

- Pre-trained models fine-tuned for a particular task or domain.
- They may also be trained from scratch when enough labeled data is available.

Key points

- Use task-specific data and objectives.
- Can be specialized for different domains (e.g. medical, legal).
- Sometimes use parameter-efficient tuning (e.g. LoRA, adapters).

Model	Type	Task
BERT	Encoder	Classification, QA
GPT	Decoder	Generation, Chat
T5	Encoder-Decoder	Translation, Summarization

★ Important

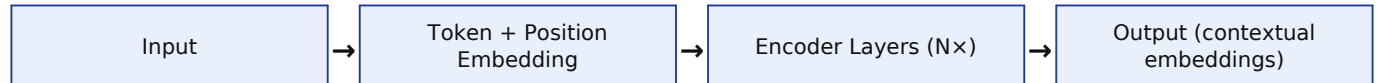
- Pre-training gives general knowledge.
- Fine-tuning adapts to a specific task.
- Transfer learning helps when data is limited.
- Self-supervised learning creates training signals without labels.
- Task-specific models are optimized for particular applications.

Transformers — Page 13

SUBTOPIC: BERT, GPT & Transformer Variants

1. BERT — Encoder-only

- BERT (Bidirectional Encoder Representations from Transformers) is an **encoder-only** model.
- It understands context from both left and right (bidirectional) using masked language modeling (MLM).
- Pre-trained on large text corpora and fine-tuned for downstream tasks.



Key points

- Encoder-only (bidirectional attention).
- Uses MLM during pre-training.
- Great for understanding context and classification tasks.
- Example: BERT-base, BERT-large.

2. GPT — Decoder-only

- GPT (Generative Pre-trained Transformer) is a **decoder-only** model.
- It uses causal (masked) self-attention to look only at previous tokens.
- Trained with next token prediction (autoregressive).

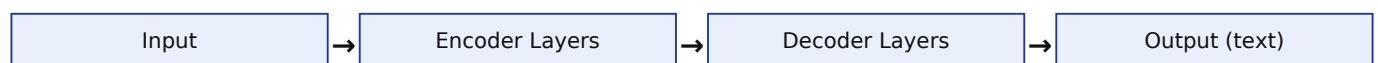


Key points

- Decoder-only (causal attention).
- Uses next token prediction during pre-training.
- Best for text generation and creative tasks.
- Example: GPT-2, GPT-3, GPT-4, GPT-4o.

3. T5 — Encoder-Decoder

- T5 (Text-to-Text Transfer Transformer) uses an **encoder-decoder** architecture.
- It frames all tasks (like translation, summarization, QA) as text-to-text problems.



Key points

- Encoder-decoder architecture.
- Uses relative position bias (instead of absolute).
- Text-to-text framework for multiple tasks.
- Example: T5-base, T5-large.

4. RoBERTa and other variants

- RoBERTa is an improved version of BERT.
- Key changes: removed next sentence prediction (NSP), used dynamic masking, larger batch sizes, more training steps and data.
- Other variants include ALBERT, DistilBERT, ELECTRA, DeBERTa, etc. Each brings improvements in efficiency, training or performance.

Comparison of popular variants

Model	Main idea / Improvement	Architecture
RoBERTa	No NSP, dynamic masking, larger data / batch size	Encoder-only
ALBERT	Parameter sharing, factorized embeddings	Encoder-only
DistilBERT	Knowledge distillation, smaller model	Encoder-only
ELECTRA	Replaced token detection	Encoder-only
DeBERTa	Disentangled attention, better positional encoding	Encoder-only

Key points

- Different training objectives and optimizations.
- Aim to improve performance, efficiency or both.
- Still mostly encoder-only (except some hybrid models).

5. When to use each architecture?

Model	When to use?
BERT	Text understanding, classification, QA, sentiment analysis, document analysis.
GPT	Text generation, creative writing, summarization, chatbots, assistance, code generation.
T5	General purpose (translation, summarization, QA, classification, etc.) using text-to-text.
RoBERTa / Variants	When you need better performance / efficiency for understanding tasks (e.g. classification, QA) with large datasets.

Quick Comparison

- BERT → Encoder-only → Understanding
- GPT → Decoder-only → Generation
- T5 → Encoder-Decoder → Text-to-Text
- RoBERTa → Encoder-only → Improved BERT

★ Remember

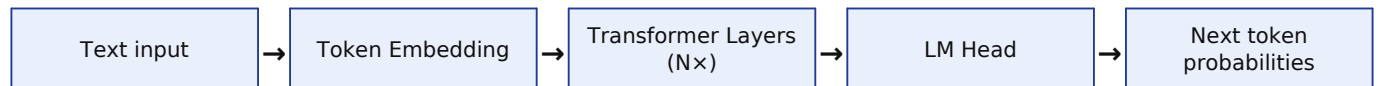
- The best model depends on your task, data size, and performance vs. efficiency trade-off.

Transformers — Page 14

SUBTOPIC: Transformers in LLMs & Generative AI

1. Large Language Models (LLMs)

- LLMs are transformer-based models trained on massive text data (internet, books, code, etc.).
- They learn language patterns, world knowledge and can generate human-like text.
- Examples: GPT-3/4, LLaMA, PaLM, Claude, etc.



Key points

- Trained with self-supervised learning (next token prediction).
- Very large number of parameters (billions).
- Learns general language understanding + generation.

2. Text Generation

- The process of generating coherent, contextually relevant text from a given prompt.
- Uses the learned patterns from training data.
- Can be used for chat, summarization, code generation, etc.



Key points

- Works one token at a time (autoregressive).
- Quality depends on prompt and model size.
- Can be controlled using parameters (temperature, top-k, etc.).

3. Context Window

- The maximum number of tokens the model can consider at once.
- Includes both input prompt + generated tokens.
- Larger context window = better long-context understanding.

Typical sizes (approx.)

Model	Context window (approx.)
GPT-3.5	4K tokens
GPT-4	8K - 128K tokens
Claude	100K+ tokens
LLaMA 3	8K - 128K tokens

★ Important

- If the context exceeds the limit, older tokens may be truncated or lost (depending on the model's strategy).

Key points

- Larger context helps with better reasoning and consistency.
- Context can be increased using longer input windows or techniques like RAG (Retrieval Augmented Generation).

4. Prompting

- The technique of giving instructions or examples to the model to get the desired output.
- Works well with few-shot or zero-shot examples.
- Can be used for tasks like summarization, translation, Q&A, etc.

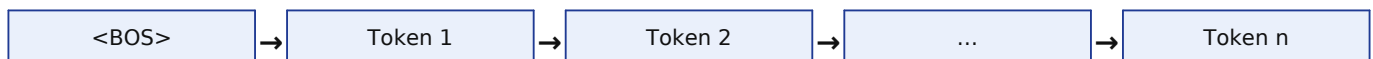
Type of prompting	Meaning
Zero-shot	no examples
Few-shot	few examples
Chain-of-thought	step-by-step reasoning
System prompt	set behavior / persona

Key points

- Clear and specific prompts give better results.
- Can control tone, format, style, etc.
- Helps in reducing hallucinations (to some extent).

5. Autoregressive Generation

- The model generates text one token at a time.
- Each new token is predicted based on the previous tokens.
- Uses causal (masked) attention to avoid looking ahead.



Key points

- Ensures coherent and contextually relevant output.
- Used in most modern LLMs (GPT, LLaMA, etc.).
- Can be influenced by sampling techniques (temperature, top-k, top-p).

6. LLM Applications

- Chatbots / virtual assistants
- Content generation
- Summarization
- Translation
- Code generation
- Question answering
- Search & RAG
- Creative writing



★ Key takeaways

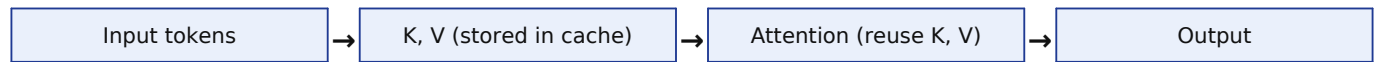
- LLMs + transformers = powerful text generation.
- Context window limits how much the model can remember.
- Prompting helps guide the model's behavior.
- Autoregressive generation enables coherent output.
- Widely used in real-world applications like chat, code, search, etc.

Transformers — Page 15

SUBTOPIC: Advanced Transformers & Interview Concepts

1. KV Cache

- KV cache stores the key and value tensors from previous tokens during inference.
- Reuses them for future tokens instead of recomputing.
- Reduces computation and speeds up generation.



Key points

- Mainly used in autoregressive generation.
- Saves memory and compute.
- For long sequences, cache size grows ($\propto$ sequence length).

2. Attention Complexity

- Self-attention has **quadratic complexity** w.r.t. sequence length.
- Each token attends to all other tokens.

Time: $O(n^2 \cdot d)$ Space: $O(n^2)$ (n = sequence length, d = hidden size)

Optimizations

- Flash Attention (reduce memory access).
- Sparse attention (attend to subset).
- Linear attention (approximation).

Method	Time Complexity	Space Complexity
Standard Attention	$O(n^2)$	$O(n^2)$
Flash Attention	$O(n^2)$	$O(n)$
Linear Attention	$O(n)$	$O(n)$

Key point

- Attention is the main computational bottleneck in long sequences.

3. Long-Context Transformers

- Models that can handle very long input sequences (e.g. 32K, 128K).
- Requires architectural and training changes.
- Helps in tasks like document understanding, code analysis, etc.

Techniques

- RoPE / ALiBi (better positional encoding).
- Sparse / sliding window attention.
- LongRoPE, YaRN (better extrapolation).

Example

- GPT-4o $\rightarrow$ 128K; Claude $\rightarrow$ 200K+
- Helps in understanding long documents, videos, code, etc. $\rightarrow$ better context understanding and reasoning.

4. Flash Attention

- An optimized attention algorithm.
- Reduces memory usage and improves speed.
- Avoids storing the full attention matrix.

How it works?

- 1. Computes attention in blocks (tiling).
- 2. Uses efficient GPU memory access.
- 3. Recomputes values instead of storing all intermediate results.

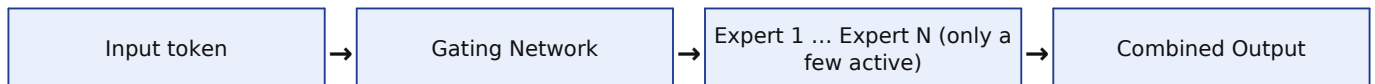


Key points

- 2–4× faster than standard attention.
- Significantly less memory usage.
- Widely used in modern LLMs (e.g. Llama 3, Mistral).

5. Mixture of Experts (MoE)

- Uses multiple expert networks instead of one large model.
- A gating network selects relevant experts for each token.
- Improves efficiency and scalability.



Benefits

- Better scalability.
- Higher performance with fewer active parameters.
- Used in models like Switch Transformer (Google).

Key idea

- Only a few experts are activated per token.

6. Quantization

- Reduces model size and memory usage by using lower precision.
- E.g. FP32 → INT8 / INT4 / FP16.
- Minimal impact on performance (with proper calibration).

Precision	Size Reduction	Typical Use
FP32	1×	Training / High accuracy
FP16	2×	Mixed precision training
INT8	4×	Inference (common)
INT4	8×	Edge / Mobile / Faster inference

- Techniques: Post-training quantization (PTQ), Quantization-aware training (QAT), GPTQ / AWQ.

Key point

- Quantization enables deployment on resource-constrained devices.

7. Transformer Optimization

- Mixed precision training (FP16 / BF16).
- Gradient accumulation (larger batch size).
- Gradient checkpointing (save memory).
- Better optimizers (AdamW, Lion).

- Efficient attention (Flash Attention, sparse).

★ **Remember**

- Focus on the concept, not just memorizing facts.
- Be ready to explain trade-offs and real-world use cases.
- Draw diagrams when needed.

8. Common Transformer Interview Questions

1. Why do we use positional encoding in Transformers?
2. What is the difference between encoder-only, decoder-only and encoder-decoder?
3. How does self-attention work? Explain with an example.
4. What is the role of layer normalization and residual connections?
5. How does Flash Attention improve performance?
6. How are MoE models? Why are they useful?
7. How does quantization affect model performance?
8. Compare BERT, GPT and T5. When would you use each?
9. How do long-context models handle very long inputs?
10. What are the trade-offs in model optimization techniques?